



THE ICFAI UNIVERSITY MIZORAM

Department of Computer Applications

FORMEASE

Premium Online Application Assistance Platform
Your Forms. Simplified.

A Minor Project Report submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Submitted by

KC VANLALHRIATA

Registration No: 24STUCMZD01358

Department of Computer Applications (BCA)

The ICFAI University Mizoram

Durtlang North, Aizawl, Mizoram — 796025

June 2026

DECLARATION

I, **KC Vanlalhriata**, bearing Registration Number **24STUCMZD01358**, a student of the Bachelor of Computer Applications (BCA) programme at The ICFAI University Mizoram, hereby declare that the minor project entitled “**FormEase — Premium Online Application Assistance Platform**” has been carried out by me under the guidance of my project guide, and that this report represents my original work.

I further declare that the project has not been submitted, either in part or in full, for the award of any degree or diploma in this or any other university or institution. All sources of information and tools used during the development of this project have been duly acknowledged in the bibliography.

Place: Aizawl, Mizoram

Date: June 2026

KC Vanlalhriata

Registration No: 24STUCMZD01358

Department of Computer Applications (BCA)

CERTIFICATE

This is to certify that the minor project report entitled “**FormEase — Premium Online Application Assistance Platform**”, submitted by **KC Vanlalhriata** (Registration No: **24STUCMZD01358**) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Computer Applications**, is a record of bona fide work carried out by the student under my supervision and guidance.

The project, in my opinion, is worthy of consideration for the award of the degree in accordance with the regulations of The ICFAI University Mizoram.

Project Guide

Department of Computer Applications

Head of the Department

Department of Computer Applications

External Examiner

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the support and encouragement of several people, to whom I owe my sincere gratitude.

I express my heartfelt thanks to my **project guide** and the faculty of the **Department of Computer Applications, The ICFAI University Mizoram**, for their valuable guidance, constructive suggestions and constant encouragement throughout the development of this project.

I am grateful to the university for providing the academic environment and resources necessary to undertake this work. I also thank my family and friends for their patience and moral support during the course of this project.

Finally, I acknowledge the open-source communities behind React, FastAPI, MongoDB, Three.js and the many other tools that made this project possible, as well as the documentation of Razorpay and the Emergent platform.

KC Vanlalhriata

ABSTRACT

FormEase is a production-quality, full-stack web platform that provides professional assistance for three common Indian application workflows — Scholarship applications, PAN Card applications and Learner's Licence applications — at a transparent, fixed per-service fee. The platform addresses a real and widespread problem: ordinary applicants frequently struggle with complex government and institutional application forms, document requirements and unclear status communication.

The system guides customers through a validated six-step application wizard with secure document uploads (drag-and-drop, type and size validation), processes payments through the Razorpay payment gateway with mandatory server-side signature verification, generates a unique trackable Application ID (FE-YEAR-XXXXX), and notifies the administrator and customer by email. A role-based administrative dashboard allows staff to review applications, verify documents, request replacements, manage application statuses, record private notes and monitor revenue through analytics charts.

The platform is built with React 19, Tailwind CSS and React Three Fiber on the frontend; FastAPI (Python) on the backend; and MongoDB Atlas as the database. Documents are stored in managed object storage with authenticated access, and the entire application is deployed on Vercel's serverless infrastructure, making it publicly accessible. Security measures include bcrypt password hashing, JWT and cookie-based sessions, Google sign-in, role-based access control, brute-force lockout and private document URLs. Automated end-to-end testing was performed at every stage of development.

Keywords: Full-stack web application, FastAPI, React, MongoDB, Razorpay, application assistance, serverless deployment, Vercel.

TABLE OF CONTENTS

Chapter 1 — Introduction

1.1 Overview | 1.2 Problem Statement | 1.3 Objectives | 1.4 Scope

Chapter 2 — System Analysis

2.1 Existing System | 2.2 Proposed System | 2.3 Feasibility Study | 2.4 System Requirements

Chapter 3 — System Design

3.1 Architecture | 3.2 Database Design | 3.3 API Design | 3.4 UI Design | 3.5 Data Flow | 3.6 Security Design

Chapter 4 — Technology Stack

4.1 Frontend | 4.2 Backend | 4.3 Database | 4.4 Third-Party Integrations | 4.5 Deployment

Chapter 5 — Implementation

5.1 Authentication | 5.2 Application Wizard | 5.3 Document Uploads | 5.4 Payments | 5.5 Tracking | 5.6 Notifications | 5.7 Admin Dashboard | 5.8 Security

Chapter 6 — Testing

6.1 Testing Approach | 6.2 Test Cases | 6.3 Test Results

Chapter 7 — Conclusion and Future Scope

Bibliography

CHAPTER 1: INTRODUCTION

1.1 Overview

FormEase is an independent online application assistance service. In India, three application workflows affect millions of people every year: applying for educational scholarships, applying for a PAN (Permanent Account Number) card from the income-tax system, and applying for a Learner's Licence before obtaining a driving licence. Although the underlying authorities provide their own portals, a large section of applicants — students, first-time internet users and busy working people — find the forms confusing, the document requirements unclear and the status communication poor.

FormEase positions itself as a trustworthy middle layer: a premium, simple and secure platform where a customer creates an account, chooses one of the three services, completes a guided multi-step form, uploads supporting documents, pays a fixed assistance fee online, and receives a unique Application ID with which the progress of the application can be tracked publicly at any time. An administrative back-office allows the operator to process applications, verify documents and communicate status changes.

1.2 Problem Statement

Existing application portals are frequently criticised for dense government-style forms, cryptic validation errors, uncertainty about required documents and a lack of human-readable status updates. Applicants often do not know whether their application was received, whether their documents were acceptable, or what happens next. There is a clear need for a service that simplifies the preparation of such applications, keeps the applicant informed, and charges a small, transparent fee for the assistance — without pretending to be a government authority.

1.3 Objectives of the Project

- To design and develop a complete full-stack web platform for online application assistance covering Scholarship, PAN Card and Learner's Licence services.
- To implement a validated multi-step application wizard with secure document upload and review-before-payment flow.
- To integrate a real payment gateway (Razorpay) with mandatory server-side payment verification and per-service pricing.
- To generate unique, human-readable Application IDs and provide public application tracking.
- To provide a secure, role-based admin dashboard for application processing, document verification, status management and revenue analytics.
- To implement automated email notifications and an extensible WhatsApp notification architecture.
- To apply production-grade security practices (password hashing, session management, access control, private document storage) and to deploy the system on a public cloud platform.

1.4 Scope of the Project

The platform covers the assistance workflow only: it helps customers complete and prepare applications and does not itself issue PAN cards, licences or scholarships, which remain the sole responsibility of the respective authorities. The current version supports three services with fixed fees (Scholarship Rs. 100, PAN Card Rs. 100, Learner's Licence Rs. 350), customer and admin roles, and

English-language UI. The system is designed so that additional services, notification channels and payment methods can be added with minimal change.

1.5 Organization of the Report

Chapter 2 analyses the existing and proposed systems and the feasibility of the project. Chapter 3 presents the system design, including architecture, database and API design. Chapter 4 describes the technology stack. Chapter 5 details the implementation of each module. Chapter 6 reports the testing strategy and results. Chapter 7 concludes and outlines future enhancements.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

At present, an applicant must interact directly with separate, unrelated portals for each service: scholarship portals of various authorities, the PAN application portals, and the Parivahan Sarathi portal for learner's licences. Each has its own account system, form structure, document format rules and payment process. Status information is typically limited to a single reference number with little explanation.

2.2 Limitations of the Existing Approach

- Complex, non-uniform forms that confuse first-time applicants.
- No single place to manage different applications.
- Unclear document requirements leading to rejections.
- Minimal status communication after submission.
- Payment and refund processes that are difficult to trace.

2.3 Proposed System

FormEase proposes a unified assistance platform. A single account manages all three service types. The application is broken into small, validated steps (Personal Details → Application Details → Documents → Review → Payment → Submitted) so the user is never confronted with a long, intimidating form. Documents are uploaded with a modern drag-and-drop interface that validates file type and size before upload. Payment is a fixed, clearly displayed fee processed by Razorpay and verified on the server before the application is accepted. Every successful application receives a unique Application ID that can be tracked on a public page. The administrator works from a dedicated dashboard with statistics, filters, document review and private notes.

2.4 Feasibility Study

Technical feasibility. The system is built entirely with mature, widely adopted open-source technologies (React, FastAPI, MongoDB) and managed services (Razorpay, object storage, email). All components were verified working in a production deployment.

Economic feasibility. The platform was developed and deployed using free-tier services (Vercel, MongoDB Atlas) and incurs no infrastructure cost at demonstration scale. Revenue is generated per application through the assistance fee.

Operational feasibility. The customer workflow requires only a phone or computer with a browser; no training is needed. The admin dashboard is equally approachable for a single operator.

2.5 System Requirements

Category	Requirement
Client (customer)	Any modern browser (Chrome, Firefox, Safari, Edge) on phone, tablet or desktop
Server (development)	Python 3.10+, Node.js 20+, MongoDB 6+
Hosting (production)	Vercel (frontend + serverless API), MongoDB Atlas M0 (database)

Category	Requirement
Key backend libraries	FastAPI, Motor (async MongoDB driver), PyJWT, bcrypt, Razorpay SDK, httpx
Key frontend libraries	React 19, Tailwind CSS, shadcn/ui, Framer Motion, React Three Fiber, Recharts, Axios

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

FormEase follows a three-tier architecture. The **presentation tier** is a React single-page application served as static files. The **application tier** is a FastAPI REST API running as a serverless Python function under the /api path on the same domain. The **data tier** comprises MongoDB Atlas (users, applications, payments, sessions, notifications, audit logs) and a managed object-storage service that holds uploaded documents. Third-party services (Razorpay for payments, the email provider for notifications, Emergent-managed Google sign-in) are invoked only from the backend so that secrets never reach the browser.

3.2 Database Design

MongoDB collections and their principal fields:

Collection	Principal Fields	Purpose
users	name, email (unique), phone, password_hash, role, picture, created_at	Customer and admin accounts
applications	application_id (unique), user_id, service_type, applicant_data, documents[], status, payment_status, admin_notes[], status_history[], created_at, updated_at	Application records and lifecycle
payments	application_id, order_id, payment_id, amount, currency, status, mode, verified_at	Razorpay order/payment records
user_sessions	user_id, session_token, expires_at	Google sign-in sessions
notifications	application_id, type, channel, recipient, message, status, created_at	Email/WhatsApp notification log
audit_logs	admin_id, action, application_id, detail, at	Admin action audit trail
counters	seq (per year)	Atomic Application ID sequence

Application documents are embedded as sub-documents inside the applications collection (doc_type, label, file_name, storage_path, size, verified, replacement_requested), while the binary files themselves are kept in object storage — following the rule of never storing large binary data in the database.

3.3 API Design

Endpoint	Method	Access	Function
/api/auth/register, /login, /logout	POST	Public	Account lifecycle (JWT httpOnly cookies)
/api/auth/me, /auth/refresh	GET/POST	Authenticated	Session info and renewal
/api/auth/forgot-password, /reset-password	POST	Public	Token-based password reset

Endpoint	Method	Access	Function
/api/auth/google/session	POST	Public	Exchange Google session for app session
/api/applications	POST	Owner	Create/resume draft application
/api/applications/{id}	GET/PATCH	Owner	Read or save progress
/api/applications/track/{id}	GET	Public	Safe status tracking only
/api/documents/upload, /documents/{...}	POST/GET/DELETE	Owner/Admin	Object-storage document handling
/api/payments/create-order, /verify	POST	Owner	Razorpay order + signature verification
/api/payments/webhook	POST	Webhook	Razorpay event confirmation
/api/admin/stats, /applications, /analytics, /notifications	GET	Admin	Dashboard data
/api/admin/applications/{id}/status, /notes, /documents/{doc}	PATCH/POST	Admin	Processing actions

3.4 User Interface Design

The interface follows a premium, minimal design language (deep navy, royal blue and soft gray palette; Outfit and Manrope typefaces) with a lightweight interactive 3D hero scene built with React Three Fiber, subtle scroll-reveal animations, and full mobile responsiveness with reduced 3D effects and large touch targets on small screens. Forms prioritise clarity: clear labels, inline validation messages, and a persistent step indicator in the application wizard.

3.5 Data Flow

Customer journey: Landing page → choose service → register/login → six-step wizard (data saved per step) → document upload to object storage → review summary → Razorpay checkout → server-side verification → Application ID issued → email notification → public tracking.

Admin journey: Email/WhatsApp notification of a new paid application → admin dashboard → application detail → document verification / replacement request → status transitions (Submitted → Documents Under Review → Processing → Completed, or Need More Information) → customer notified of each change.

3.6 Security Design

Security was treated as a first-class requirement because applicants upload sensitive identity documents: bcrypt password hashing, JWT access/refresh tokens and Google sessions in httpOnly secure cookies, role-based access control enforced on the server, login brute-force lockout (5 attempts / 15 minutes), authenticated-only document downloads (owner or admin), file type/size validation on both client and server, input sanitisation, security HTTP headers, admin audit logging, and all secrets kept in server-side environment variables. Payment acceptance depends solely on Razorpay signature verification — never on frontend-supplied claims.

CHAPTER 4: TECHNOLOGY STACK

4.1 Frontend

- **React 19** — component-based UI library; single-page application with React Router 7.
- **Tailwind CSS + shadcn/ui** — utility-first styling and an accessible component library.
- **React Three Fiber / Three.js** — the abstract 3D hero scene (floating documents, ID card, shield, check mark) with mouse parallax; simplified automatically on mobile and for reduced-motion users.
- **Framer Motion** — scroll reveals, page and wizard-step transitions, success animation.
- **Recharts** — responsive analytics charts in the admin dashboard.
- **Axios** — API client with cookie-based credentials.

4.2 Backend

- **FastAPI (Python)** — high-performance async REST API framework with automatic validation via Pydantic.
- **Motor** — asynchronous MongoDB driver.
- **PyJWT + bcrypt** — token issuance/verification and password hashing.
- **Razorpay Python SDK** — order creation and payment signature verification.
- **httpx / requests** — outbound calls to notification and storage services.

4.3 Database

MongoDB Atlas — a managed, cloud-hosted MongoDB cluster (free M0 tier) storing all application data with unique indexes on user email and application ID.

4.4 Third-Party Integrations

- **Razorpay** — payment gateway; checkout on the frontend, order creation and signature verification on the backend (test mode during development, live keys supported).
- **Managed object storage** — persistent document storage with all access proxied through authenticated backend endpoints.
- **Managed email (Resend-compatible)** — transactional emails for application received, payment confirmation, status updates and document requests.
- **Emergent-managed Google Auth** — one-click Google sign-in alongside email/password accounts.
- **WhatsApp Cloud API architecture** — notification service implemented with a mock sender, ready for credentials.

4.5 Deployment

The application is deployed on **Vercel**: the React frontend is served as static files and the FastAPI backend runs as a serverless Python function, both under a single public domain (hriata.vercel.app). This serverless approach removes server maintenance entirely and scales automatically. The database runs on MongoDB Atlas.

CHAPTER 5: IMPLEMENTATION

5.1 Authentication Module

Customers register with name, email, mobile and password; passwords are hashed with bcrypt. Login issues short-lived JWT access tokens and long-lived refresh tokens in httpOnly, secure cookies (never readable by JavaScript). A Google sign-in option uses the managed OAuth flow: the backend exchanges the provider session server-side, creates or links the account by email, and issues the same application session. Five failed logins lock the account+IP pair for fifteen minutes. Password reset uses single-use, one-hour tokens delivered by email.

5.2 Application Wizard Module

Selecting a service creates (or resumes) a draft application with its unique ID already assigned. The wizard presents six steps — Personal Details, Application Details (service-specific fields), Documents, Review, Payment and Submitted — with per-field validation (required fields, email format, 10-digit mobile, 6-digit PIN), progress saving on every step, and free navigation back to edit earlier steps.

5.3 Document Upload Module

Each service defines a document checklist with Required/Optional badges (for example, PAN requires identity proof, address proof and a photograph). The uploader supports drag-and-drop and browsing, shows progress, previews images, and allows replace and delete. Files are limited to JPG/PNG/PDF and 5 MB, validated on both client and server. Files are stored in managed object storage under a randomised path and can only be read through an authenticated backend endpoint that checks ownership or admin rights.

5.4 Payment Module

The fee is computed on the server from the service type (Scholarship Rs. 100, PAN Rs. 100, Learner's Licence Rs. 350) so it cannot be manipulated from the browser. The backend creates a Razorpay order; the customer completes payment in Razorpay's secure checkout (UPI including Google Pay, cards, net banking); the backend then verifies the Razorpay signature before marking the application paid and submitted. A webhook endpoint is provided for payment confirmation events. A clearly labelled demo mode exists for safe testing when live keys are absent.

5.5 Application ID and Tracking Module

Application IDs follow the format FE-YEAR-XXXXX (e.g., FE-2026-00001) generated by an atomic database counter, guaranteeing uniqueness. The public tracking page shows a five-stage timeline (Submitted → Payment Confirmed → Documents Received → Processing → Completed) plus service, dates and payment status — without exposing any personal data or documents.

5.6 Notification Module

A backend notification service sends transactional emails (to the admin on every new paid application; to the customer on receipt, status change and document requests) and records every message in a notification log visible in the admin dashboard. A WhatsApp Cloud API channel is implemented in the same service and activates as soon as business credentials are configured; until then it operates in a clearly logged mock mode.

5.7 Admin Dashboard Module

The role-gated /admin area provides overview statistics (total/today/pending/completed applications, revenue), four responsive charts (applications and revenue over time, applications by service, status distribution), a filterable and searchable applications table, and a detail view for each application with customer data, payment record, secure document viewing, verify / request-replacement actions, status management and private notes (never shown to customers). Every admin action is written to an audit log.

5.8 Security Implementation

All private and admin endpoints are protected by authentication and role checks on the server. Document files have no public URL. Security headers are applied to every response. Secrets live only in environment variables; none appear in the frontend bundle or the code repository, which is kept clean of environment files via .gitignore.

CHAPTER 6: TESTING

6.1 Testing Approach

Testing combined automated API testing (pytest against the running backend), browser-based end-to-end testing of the real user interface (registration through payment and admin processing), and security-oriented negative testing. Every major feature and bug fix was verified by an independent testing pass before being accepted. The production deployment on Vercel was additionally verified live after each infrastructure change.

6.2 Test Cases (Representative Sample)

N o.	Test Case	Expected Result	Status
1	Register with valid details	Account created; session cookie set	Pass
2	Login with wrong password	401 error, attempt counted	Pass
3	Five consecutive failed logins	429 lockout for 15 minutes	Pass
4	Upload .exe file as document	Rejected with type error	Pass
5	Upload 6 MB file	Rejected with size error	Pass
6	Payment with tampered signature	400 rejected; application not marked paid	Pass
7	Create order with amount in request body	Server ignores body; charges service fee	Pass
8	Track valid / invalid Application ID	Timeline shown / 404	Pass
9	Access another user's application	403 Forbidden	Pass
10	Access document URL without login	401 Unauthorized	Pass
11	Access /admin endpoints as customer	403 Forbidden	Pass
12	Per-service order amounts	Rs. 100 / Rs. 100 / Rs. 350 orders on gateway	Pass
13	Admin status change	History recorded; customer emailed	Pass
14	Google sign-in session exchange	Session created; bogus ID rejected	Pass

6.3 Test Results

All backend test suites passed (21/21 API tests in the core suite, 12/12 in the pricing suite, 9/9 in the Google-auth suite, 10/10 in the object-storage suite). Browser end-to-end passes confirmed the complete customer and admin journeys, and the final production verification on the live Vercel deployment confirmed registration, application submission, document upload, signed payment, tracking, email delivery and admin processing — all operational.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

FormEase demonstrates that a genuinely production-grade service platform can be designed, built, tested and deployed as a minor project. The system delivers the complete intended experience: a premium public website, a guided application workflow, secure document handling, real online payments with server-side verification, public tracking, automated notifications and a capable administrative back-office — all publicly accessible on free-tier cloud infrastructure. Beyond the functional result, the project applied professional engineering practices throughout: environment-based configuration, security-first design, automated testing at every step, and incremental verified deployments.

7.2 Future Enhancements

- Switch Razorpay to live keys to accept real payments (the code path is already production-ready).
- Activate WhatsApp Cloud API notifications by adding business credentials.
- Add more assistance services using the existing service-definition structure.
- Introduce SMS notifications and in-app notification centre.
- Add downloadable PDF receipts for payments.
- Connect a custom domain and move to paid tiers for always-warm performance.

BIBLIOGRAPHY

- React documentation — <https://react.dev>
- FastAPI documentation — <https://fastapi.tiangolo.com>
- MongoDB Manual — <https://www.mongodb.com/docs>
- Razorpay API documentation — <https://razorpay.com/docs>
- Three.js / React Three Fiber documentation — <https://threejs.org>, <https://docs.pmnd.rs>
- Tailwind CSS documentation — <https://tailwindcss.com>
- Vercel documentation — <https://vercel.com/docs>
- Flask- and Django-alternative comparisons and REST design references — various online sources